

ACTIVIDAD DE CLASE 2

FLUJO DE USUARIO Y DISEÑO DE INTERFACES

Aplicaciones Móviles · Trabajo grupal · Material Design 3 + IA

Propósito de la actividad

Transformar el MVP definido en la Clase 1 en una propuesta visual de aplicación móvil. Al finalizar, el equipo debe tener definido el nombre de la app, su identidad visual, un flujo de usuario representado mediante UML y las interfaces principales que posteriormente serán implementadas en Android.

Nombre del proyecto/caso: AquaCheck Buceo**Sección:** 002D**Nombre Equipo y número:** Los Chiikawitas - Grupo 8**Fecha:** 10-09-2026**Integrantes**

Nombre y apellido	Rol dentro del equipo
Felipe Barra	Lider
Aolani Caiguan	Backend
Renata Orellana	FrontEnd

Entrega

- Completen este mismo documento durante la clase. No creen un documento distinto.
- Guarden el archivo con el nombre: Evidencia_Clase_02_Disenio_EquipoXX.docx.
- Suban este documento a docs/evidencias/clase-02/ y los recursos visuales a docs/disenio/.
- Actualicen el README.md e incluyan el flujo de usuario UML. Realicen un commit identificable.

1. Identidad visual de la aplicación

Antes de diseñar las pantallas, definan una identidad mínima para la aplicación. Esta identidad debe mantenerse en todas las interfaces del proyecto.

1.1 Nombre de la aplicación

Definan un nombre breve, fácil de recordar y relacionado con el problema que resolverá la aplicación.

Nombre definitivo de la aplicación
AquaCheck

1.2 Logotipo

Diseñen o generen un logotipo con transparencia en formato .png o svg simple. Puede utilizarse una herramienta de IA como chat GPT en imágenes, pero la versión final debe ser seleccionada y validada por el equipo.

Elemento	Definición del equipo
Nombre del archivo	Logotipo de AquaCheck
Ubicación en el repositorio	docs/disenio/logo.png
¿Qué representa el logotipo?	Representa la seguridad y la verificación (el check) fusionadas con el entorno marino (la gota/ola), transmitiendo confianza al buceador antes de su inmersión.

1.3 Paleta de colores

Definan entre 3 y 6 colores y registren su código hexadecimal. Piensen desde ahora qué función tendrá cada color dentro de la interfaz. [Leer esto antes de escoger los colores.](#)

Color	Código HEX	Uso dentro de la app
Principal	#005B96	TopAppBar, botones principales y elementos de énfasis
Secundario	#03A9F4	Iconos activos, Floating action button y detalles visuales
Fondo	#F5F5F5	Color de fondo general de las pantallas para reducir la fatiga visual bajo el sol
Texto	#212121	Titulos, subtítulos y textos general para asegurar legibilidad y alto contraste
Adicional	#D32F2F	Mensajes de error, items del checklist faltantes

1.4 Justificación de la identidad

Justificación breve de la paleta de colores:

La paleta está inspirada en el océano (tonos azules) para conectar inmediatamente con el buceo. Se priorizó un fondo claro con texto oscuro y colores de acento vibrantes para garantizar una alta legibilidad (accesibilidad) en exteriores, considerando que la app se usará frecuentemente en botes o playas con luz solar directa. El rojo se reserva estrictamente para la seguridad.

2. Flujo de usuario mediante UML

El flujo de usuario muestra cómo una persona recorrerá la aplicación para completar una tarea. En esta clase deben reemplazar el flujo simple de cajas y flechas de la Clase 1 por un Diagrama de Actividad UML.

Recuerden

El Diagrama de Actividad UML debe utilizar inicio y fin, actividades, flechas de secuencia y decisiones cuando existan caminos alternativos. No es necesario representar código ni clases.

2.1 Recorrido principal

Antes de dibujar el UML, describan el recorrido principal del usuario. Esto les permitirá validar que no falten pasos o pantallas.

N°	Acción del usuario	Pantalla	¿Existe decisión?
1	Abrir app e Iniciar Sesión	Abrir app e Iniciar Sesión	Sí
2	Ver lista de bitácoras	Home / Dashboard	No
3	Presionar botón "Nuevo Checklist"	Home / Dashboard	No
4	Marcar ítems de equipamiento (Ej: tanque, regulador)	Pantalla de Checklist	Sí
5	Visualizar advertencia de seguridad si faltan ítems	Pantalla de Checklist	No
6	Guardar inmersión validada	Resumen Inmersión	No
7	Volver al inicio	Home / Dashboard	No

2.2 Diagrama de Actividad UML

Construyan el diagrama con la herramienta que prefieran. Luego exportarlo como imagen PNG para incorporarlo al proyecto.

//Pueden definir el diagrama transformarlo a código uml y generar un diagrama en [Plantext](#)

Requisito	Definición
Nombre recomendado	flujo-usuario-uml.png
Ubicación	docs/disenio/flujo-usuario-uml.png
Debe quedar visible en github con Mermaid	README.md
Debe representar	El flujo principal del MVP

2.3 Validación del flujo

- ☐ El flujo comienza y termina claramente.
- ☐ Todas las pantallas importantes del MVP aparecen en el recorrido.
- ☐ Las decisiones del usuario están representadas cuando corresponde.
- ☐ El equipo puede explicar el flujo sin hablar todavía de Kotlin o código.

3. Definición de las pantallas del MVP

Utilicen el MVP definido en la Clase 1 y el flujo UML para decidir qué pantallas realmente necesita la aplicación. No agreguen pantallas que no respondan a un requerimiento.

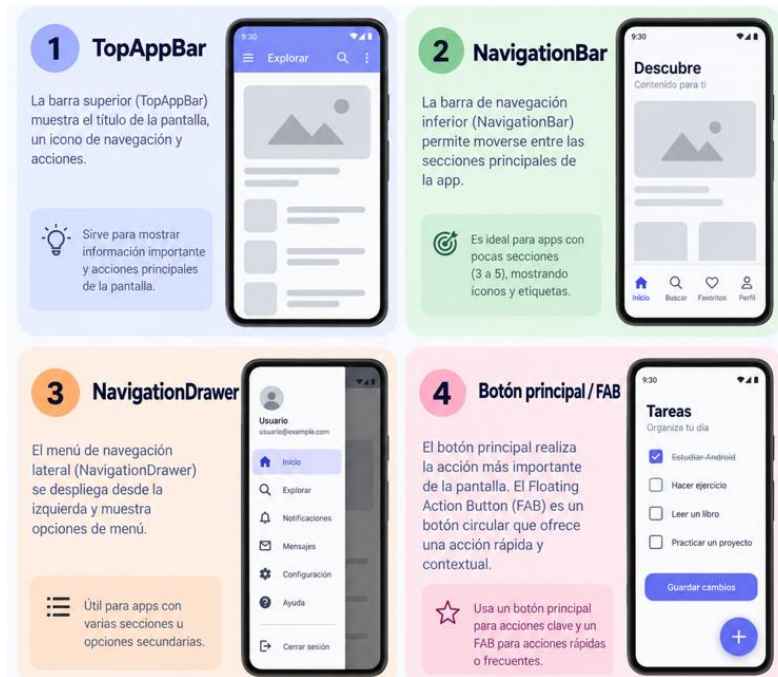
3.1 Inventario de pantallas

Pantalla	Objetivo	Usuario principal	Acción principal
Login / Registro	Autenticar al usuario para mantener su bitácora	Buceador	Ingresar correo y contraseña
Home (Dashboard)	Mostrar inmersiones recientes y acceso a la acción principal	Buceador	Iniciar un nuevo checklist de buceo
Checklist de Equipamiento	Asegurar que el usuario lleve todo el equipo seguro	Buceador	Marcar Checkboxes(Aire, Regulador, Máscara, etc.).
Resumen Inmersión	Confirmar que la validación fue exitosa y guardar el registro	Buceador	Presionar “Finalizar y Guardar”

3.2 Navegación principal

Definan cómo accede el usuario a las funciones principales.

No es obligatorio utilizar todos los tipos de navegación; seleccionen el que tenga sentido para su aplicación.}



Elemento	¿Se utilizará?	¿Para qué?
NavigationBar	Sí	Para moverse rápidamente entre "Home", "Historial" y "Perfil" en la parte inferior de la pantalla.
NavigationDrawer	No	No se utilizará para mantener la interfaz ágil y requerir menos toques (gestos) del usuario
TopAppBar	Sí	Para mostrar el título de la pantalla actual (Ej: "Nuevo Checklist") y alojar el botón de retroceso (Back navigation).
Botón principal / FAB	Sí	Un Floating Action Button en la pantalla Home para la acción más importante: "Crear Nuevo Checklist".

3.3 Regla de alcance

Mantengan el MVP

Cada pantalla debe justificar su existencia mediante una funcionalidad definida en la Clase 1. Si una pantalla no aporta al MVP, déjenla fuera de esta primera versión.

4. Diseño de interfaces con apoyo de IA

Utilicen una herramienta de IA para generar una primera propuesta visual. Pueden trabajar con [Google AI Studio](#) o [Stitch](#) como herramienta principal y utilizar otras herramientas de generación de imágenes como apoyo. El objetivo es diseñar y tener una referencia visual para más adelante.

4.1 Interfaces a generar

Generen las pantallas principales definidas en la sección anterior. Guarden cada diseño como imagen y utilicen nombres de archivo simples.

Pantalla	Archivo PNG	Herramienta utilizada	Cambio realizado por el equipo
Login	login_mockup.png	Gemini	Se ajustaron los colores al azul definido en la paleta
Home	home_mockup.png	Gemini	Se movio boton principal a un FAB en la esquina inferior derecha
Checklist	checklist_mockup.png	Gemini	Se agrandaron los Checkboxes para facilitar el toque (touch target) con las manos húmedas

5. Aplicación de Material Design 3

Revisen las interfaces y relacionen los elementos visuales con componentes que posteriormente podrán implementar mediante Jetpack Compose y [Material 3](#).

Revisen el los componentes de Material 3 y definan como equipo que componentes utilizarán en cada pantalla de la aplicación

5.1 Componentes identificados

Pantalla	Componente Material 3	¿Para qué se utilizará?
Login	TextField, Button	Ingreso de credenciales de usuario y botón para confirmar el acceso.
Home	TopAppBar, FloatingActionButton, Card	Barra superior con logo, botón flotante para agregar inmersión, y tarjetas (Cards) para mostrar el resumen de buceos anteriores.
Checklist	Checkbox, Button, Snackbar	Marcar el equipo validado, botón de guardar y Snackbar para mostrar alertas si falta equipo crítico.
General	NavigationBar	Navegación persistente en la zona inferior.

Componentes de referencia

TopAppBar · NavigationBar ·
 NavigationDrawer · Card · Button ·
 FloatingActionButton · TextField · Icon
 · Dialog · Snackbar

6. Organización del repositorio

Los diseños son parte del proyecto. Guárdalos dentro del mismo repositorio para mantener trazabilidad entre análisis, diseño e implementación.

Elemento	Ubicación recomendada
Documento de evidencia	docs/evidencias/clase-02/
Logotipo	docs/disenio/logo.png
Diagrama UML	docs/disenio/flujo-usuario-uml.png
Interfaces	docs/disenio/interfaces/

7. Evidencia y actualización del README

Antes de finalizar, dejen el repositorio organizado y asegúrense de que otra persona pueda comprender el proyecto revisando el README.md.

7.1 Contenido mínimo del README.md

El README debe permitir comprender rápidamente qué aplicación construirán y cómo se organizará visualmente.

Sección del README	Contenido mínimo
Nombre y propósito	Nombre de la app y breve descripción del problema.
Identidad visual	Logotipo y paleta de colores.
Flujo de usuario	Imagen del Diagrama de Actividad UML.
Pantallas principales	Lista de interfaces y ubicación dentro de docs/diseño/.
Integrantes	Aolani, Renata, Felipe
Tecnologías	Herramientas y lenguajes a utilizar (ej. Android Studio, Kotlin, Jetpack Compose, Material Design 3).

Nombre y ubicación de la evidencia

Elemento	Definición
Nombre recomendado	Evidencia_Clase_02_Diseño_EquipoXX.docx
Carpeta de evidencia	docs/evidencias/clase-02/
Carpeta de diseño	docs/diseño/
Commit sugerido	docs: agrega flujo UML e interfaces del proyecto

Cierre de la clase

¿Un desarrollador externo podría comprender qué aplicación construirán y cómo debería verse revisando el README y la carpeta de diseño?

Respuesta: Sí. El README centraliza la identidad visual, el propósito del MVP (validar la seguridad del buceo), y muestra el flujo lógico a través del diagrama UML de Mermaid. Además, al tener las pantallas renderizadas en la carpeta `docs/diseño/`, cualquier desarrollador tiene un mapa claro tanto del comportamiento (Backend/Lógica) como de la interfaz esperada (FrontEnd/Jetpack Compose), eliminando la ambigüedad antes de escribir la primera línea de código.

CLASE 2 COMPLETADA